

POLICY LAYER SPEC v1.1.2 (Final Accepted)

BTC Futures Scalping Alert System

Indicator-agnostic Policy Layer (Deterministic)

0) 목적과 목표

0-1. 목적 (고정)

Policy Layer는 지표(FVG/OB/CHANNEL/TRENDLINE)가 아무리 완벽한 POI를 만들어도, 아래 3가지가 나쁘면 기대값이 깨지는 문제를 시스템적으로 방지한다.

- 비용/슬리피지(c)가 커서 손익분기 승률이 과도하게 요구되는 구간 차단
- 레짐(변동성/유동성/스프레드)이 나빠 같은 POI도 승률이 붕괴되는 구간 차단/감점
- 리스크(손실 누적/신호 과다/포트폴리오 노출 집중)로 계좌가 "많이 치다 죽는" 구간에서 자동 보수 모드 전환

0-2. 핵심 철학 (고정)

- Indicator-agnostic:** Policy는 지표의 내부 정의를 몰라도 동작해야 한다.
- Deterministic:** 동일 입력이면 동일 출력. 랜덤/시각의존/휴리스틱 금지.
- Hard Defense + Attack Preservation:**
 - 극단 레짐/비용/손실한도는 하드 차단(HALT/BLOCK)
 - 강한 셋업은 isExceptional 경로로 "차단 대신 강감점"까지 허용
 - 단, 하드 HALT/비용 하드블록/손실한도 HALT는 절대 override 금지

0-3. 스코어/합격선에 대한 결론 (고정)

- Policy는 외부 출력(텔레그램/알림) 합격선(PASS_THRESHOLD)을 정의하지 않는다.
- Policy는 ALLOW/BLOCK + policyScoreDeltaSum + tags/reasons + suggestedRiskPct만 반환한다.
- 최종 출력/Best1/합격선은 SignalRouter 레이어에서 고정한다.

1) 범위와 비범위

1-1. 범위 (Policy가 한다)

입력 시그널 후보(SignalCandidate)에 대해:

- RegimeGate / CostGate / RewardProxyAdjust / ConcentrationGate / EdgeEvidenceGate / RiskManager / PortfolioExposureGate 를 고정 순서로 평가
- decision(ALLOW/BLOCK), policyScoreDeltaSum, policyTags, reasons, riskMode, suggestedRiskPct 반환

1-2. 비범위 (Policy가 안 한다)

- 지표 POI 생성/무효화/레이어링(지표 엔진의 정본 영역)
- 최종 알림 라우팅/Best1/출력 제한(= SignalRouter 영역)
- 주문 실행/포지션 관리(알림 시스템 기본)

2) 입력/출력 계약 (고정)

2-1. 입력: SignalCandidate (필수)

아래 필드는 반드시 존재해야 한다. 없으면 Policy는 `BLOCK(reason=missing_required_field)` .

필수:

- symbol: string
- time: ISO-8601 UTC string (YYYY-MM-DDTHH:mm:ssZ)
- source: "FVG" | "OB" | "CHANNEL" | "TRENDLINE"
- eventType: "REACTION" | "ENTRY_WINDOW_OPEN"
- dir: "BULL" | "BEAR"
- poiTier: "D1_POI" | "H4_CORE" | "SETUP" | "OTHER"
- poild: string

가격/리스크 계산 필수:

- entryBoundaryPrice: number (게이트 기준선)
- hardInvalidationPrice: number (아이디어 붕괴 경계)
- lastPrice: number (평가 시점 최종가)
- midPrice: number (없으면 lastPrice로 대체 가능)
- tickSize: number

변동성 입력 필수:

- lta14: number (LTF ATR14, price 단위; STOP_BUFFER 계산에 사용)

트리거/콜라보 (권장; 없으면 기본값 적용 가능):

- triggerCount: 0|1|2|3 (없으면 0)
- collabStrength: "NONE"|"WEAK"|"STRONG" (없으면 "NONE")
- hasStack: boolean (없으면 false)
- tags: string[] (없으면 [])

선택 (기대값 프록시; 없어도 동작해야 함):

- expectedRR: number (reward/risk 추정치)
- tpRefPrice: number (TP 프록시 가격; expectedRR 없을 때 RR 계산에 사용)

주의(정보 고정): s_raw_bps(스탑폭) 계산의 entryRef는 entryBoundaryPrice가 아니라 midPrice(없으면 lastPrice)로 한다. (boundary와 invalidation이 같아 0이 되는 붕괴 케이스 방지)

2-2. 입력: MarketSnapshot (필수)

SignalCandidate.time과 동일한 샘플 시각으로 묶인 스냅샷.

필수:

- time: ISO UTC
- symbol: string
- bid: number
- ask: number
- last: number
- mid: number

레짐/윈도우 입력 (기본 TF=M5 권장):

- atr14_price: number (M5 ATR14 price)
- atr14_bps: number (M5 ATR14 bps)
- volume_m5: number
- barChange_bps_m5: number (M5 1봉 변화폭 bps; 없으면 0)

2-3. 입력: AccountSnapshot (필수)

필수:

- time: ISO UTC
- equity: number
- riskMode: "NORMAL" | "L1" | "L2" | "HALT" (이전 상태; RiskManager hysteresis에 사용)
- realizedPnl_24h_pct: number (rolling 24h, equity 대비 %, 예: -1.2)
- consecutiveLosses: number

선택 (포트폴리오 노출; 있으면 PortfolioExposureGate 적용):

- openRiskPct: number (0~1, 예: 0.012 = 1.2% open risk)

선택 (Router 전용; Policy는 사용하지 않음):

- signalsSent_60m: number

2-4. 출력: PolicyResult (고정)

- decision: "ALLOW" | "BLOCK"
- policyScoreDeltaSum: number
- policyTags: string[] (unique + lexicographic sort)
- reasons: string[] (unique + lexicographic sort)
- riskMode: "NORMAL" | "L1" | "L2" | "HALT"
- suggestedRiskPct: number (0~1, 예: 0.010 = 1.0%)
- derived: DerivedValues

3) 공통 상수 (전 심볼 고정 / 변경 불가)

3-1. 시간/윈도우 (고정)

- POLICY_BASE_TF = "M5"
- W_LONG_BARS = 2016 (M5 기준 7일)
- W_SHORT_BARS = 288 (M5 기준 1일)
- WINDOW_CONC_15M = 15 minutes
- WINDOW_PNL_ROLLING = 24 hours

3-2. CostGate (고정)

- SC_BLOCK = 3.0
- SC_PENALTY_1 = 4.0
- SC_PENALTY_2 = 5.0

보수적 폴백 (구성 파일이 없을 때):

- FEE_BPS_ROUNDTRIP_DEFAULT = 8.0
- SLIPPAGE_BPS_FLOOR = 1.0

3-3. RegimeGate (고정)

- SPREAD_HALT_BPS = 50
- SPREAD_CAUTION_BPS = 20
- VOL_Q_LOW = 0.20
- VOL_Q_HIGH = 0.80
- VOLSHIFT_RATIO_TH = 1.5
- VOLSHIFT_SHORT_Q = 0.95
- LIQUIDITY_Q_LOW = 0.20

3-4. StopBuffer (고정)

- STOP_BUFFER_TICKS_MIN = 2
- STOP_BUFFER_ATR_LOW = 0.08
- STOP_BUFFER_ATR_NORMAL = 0.10
- STOP_BUFFER_ATR_HIGH = 0.15

3-5. ConcentrationGate (고정)

- POI_CLUSTER_STEP_BPS = 10
- MAX_UNIQUE_CLUSTERS_15M_PER_DIR = 5
- CONC_OVERRIDE_PENALTY = -30
- CONC_DUPLICATE_PENALTY = -10

3-6. EdgeEvidenceGate (고정)

- EDGE_MIN_SAMPLES = 30
- LCB_Z = 1.28
- EDGE_NO_EVIDENCE_PENALTY = -5
- EDGE_COLDSTART_EXTRA_PENALTY = -10
- EDGE_LCB_NEG_PENALTY = -35
- EDGE_LCB_NEG_BLOCK_SC = 4.5

Kelly (기본 OFF):

- KELLY_DEFAULT_ENABLED = false
- KELLY_ENABLE_MIN_SAMPLES = 100
- KELLY_HALF = 0.5
- KELLY_MAX_MULT = 1.0

3-7. RiskManager (고정)

HALT hysteresis (rolling 24h 기준):

- HALT_ENTER_PNL_24H = -2.0
- HALT_EXIT_PNL_24H = -1.8

연속 손실 모드:

- CONSEC_LOSS_L1 = 3
- CONSEC_LOSS_L2 = 5

조기 복귀 최소 R:

- MIN_RECOVERY_R_L1 = 0.30
- MIN_RECOVERY_R_L2_SUM2WINS = 0.40

모드별 사이징 힌트 (출력만):

- RISK_PCT_NORMAL = 0.010
- RISK_PCT_L1 = 0.006
- RISK_PCT_L2 = 0.003
- RISK_PCT_HALT = 0.000

3-8. RewardProxyAdjust (고정; soft only)

expectedRR 기반 패널티 (soft):

- RR_PENALTY_LT_1_0 = -15
- RR_PENALTY_LT_1_2 = -8

fallback rewardProxy 패널티 (soft):

- REWARDPROXY_LOW_PENALTY = -10
- REWARDPROXY_MID_PENALTY = -5
- REWARDPROXY_HIGH_PENALTY = 0

3-9. PortfolioExposureGate (고정; non-overridable)

포트폴리오 총 오픈 리스크 상한 (0~1, fraction):

- PORTFOLIO_CAP_NORMAL = 0.020
- PORTFOLIO_CAP_L1 = 0.012
- PORTFOLIO_CAP_L2 = 0.006
- PORTFOLIO_CAP_HALT = 0.000

4) Derived Value Pool (단일 파생값 풀; 결정론 핵심)

4-1. DerivedValues 구조 (고정)

- spread_bps
- fee_bps_roundtrip
- slippage_bps_p95_est
- slippage_multiplier
- c_bps_roundtrip
- entryRefPrice (midPrice else lastPrice)
- s_raw_bps
- stopBuffer_price
- stopBuffer_bps
- s_effective_bps
- SC
- fastMove (barChange_bps_m5 > atr14_bps * 0.30)
- atrRatio
- q95_short
- regimeState: "OK" | "CAUTION" | "TRANSITION" | "HALT"
- volState: "LOW" | "NORMAL" | "HIGH"
- liquidityState: "NORMAL" | "LOW"
- poiClusterKey (Concentration용)
- evidenceLevel: "FINE" | "MID" | "COARSE" | "NO_EVIDENCE"
- usedSignature: "FINE" | "MID" | "COARSE" | "NONE"
- lcbR: number | null
- reward_bps: number | null
- expectedRR_used: number | null
- rewardProxy: "HIGH" | "MID" | "LOW"
- isExceptional: boolean

4-2. spread_bps (고정)

```
spread_bps = (ask - bid) / mid * 10000
```

4-3. slippage_bps_p95_est (고정)

```
slippage_bps_p95_est = max(SLIPPAGE_BPS_FLOOR, spread_bps * 0.60)
```

slippage_multiplier (고정):

- 기본 1.0
- liquidityState == LOW → * 1.5
- regimeState == TRANSITION → * 2.0
- fastMove == true → * 2.0

4-4. stopBuffer_price (고정)

volState별 STOP_BUFFER_ATR 사용: LOW → 0.08 / NORMAL → 0.10 / HIGH → 0.15

```
stopBuffer_price = max(STOP_BUFFER_TICKS_MIN * tickSize, 1 * atr14 * STOP_BUFFER_ATR_x)
```

4-5. s_raw_bps / s_effective_bps (고정)

```
entryRefPrice = (midPrice != 0 ? midPrice : lastPrice)
s_raw_bps = abs(entryRefPrice - hardInvalidationPrice) / entryRefPrice * 10000
stopBuffer_bps = stopBuffer_price / entryRefPrice * 10000
s_effective_bps = s_raw_bps + stopBuffer_bps
```

4-6. 비용 c_bps_roundtrip / SC (고정)

```
fee_bps_roundtrip = FEE_BPS_ROUNDTRIP_DEFAULT (초기 풀백; 거래소/계정별 값 있으면 교체 가능)
```

```
c_bps_roundtrip = fee_bps_roundtrip + spread_bps + slippage_bps_p95_est * slippage_multiplier
SC = s_effective_bps / c_bps_roundtrip
```

4-7. rewardProxy fallback (고정)

rewardProxy는 expectedRR/tpRefPrice가 없을 때 기대값 프록시로만 사용한다.

```
HIGH: (poiTier == D1_POI) AND (SC >= 5.0)
MID:  ((poiTier == H4_CORE) AND (SC >= 4.0)) OR ((hasStack == true) AND (SC >= 4.0))
LOW:  else
```

5) isExceptional (고정)

5-1. isExceptional 정의 (고정)

아래 둘 중 하나면 `isExceptional = true`

A) ENTRY_WINDOW_OPEN 예외 (기준)

- `eventType == "ENTRY_WINDOW_OPEN"`
- `AND (poiTier == "D1_POI" OR hasStack == true OR collabStrength == "STRONG")`
- `AND triggerCount >= 2`

B) REACTION 예외 (매우 좁게; 고정)

- `eventType == "REACTION"`
- `AND poiTier == "D1_POI"`
- `AND collabStrength == "STRONG"`
- `AND triggerCount >= 2`

5-2. override 금지 (하드 방어; 고정)

아래는 isExceptional이어도 **절대 override** 불가:

- RegimeGate가 HALT
- CostGate가 SC < 3.0 하드블록
- RiskManager가 HALT (rolling 24h loss hysteresis 포함)
- PortfolioExposureGate의 cap 초과/trim 규칙 (non-overridable)

6) 평가 파이프라인 (순서 고정)

PolicyEvaluate(signal, market, account)의 고정 순서:

- 1) **computeDerived(signal, market, account)** (단 1회)
- 2) **RegimeGate(derived)**
- 3) **CostGate(derived)**
- 4) **RewardProxyAdjust(signal, derived)** (soft only)
- 5) **ConcentrationGate(signal, derived)**
- 6) **EdgeEvidenceGate(signal, derived)**
- 7) **RiskManager(account, derived)** (riskMode/suggestedRiskPct 산출)
- 8) **PortfolioExposureGate(account, derived)** (openRiskPct 기반 cap/trim/block)
- 9) **finalize:** tags/reasons unique+sort, result 반환

7) RegimeGate v1.1.2 (고정)

7-1. 출력 (고정)

- `regimeState:` OK | CAUTION | TRANSITION | HALT
- `volState:` LOW | NORMAL | HIGH
- `liquidityState:` NORMAL | LOW

7-2. HALT (고정)

```
if spread_bps > SPREAD_HALT_BPS
-> regimeState = HALT
-> decision = BLOCK(reason=REGIME_SPREAD_HALT)
```

7-3. volState (고정)

- `atr14_bps <= q(VOL_Q_LOW, W_LONG)` → LOW
- `atr14_bps >= q(VOL_Q_HIGH, W_LONG)` → HIGH
- `else` → NORMAL

7-4. volShift 단계화 (고정)

```
atrRatio = atr14_bps_now / mean(atr14_bps, W_LONG)
q95_short = q(VOLSHIFT_SHORT_Q, W_SHORT)

shiftStrong = (atrRatio >= 1.5) AND (atr14_bps_now >= q95_short)
shiftWeak = ((atrRatio >= 1.5) OR (atr14_bps_now >= q95_short)) AND NOT shiftStrong
```

처리:

- shiftStrong → regimeState = TRANSITION
- shiftWeak → regimeState = max(regimeState, CAUTION)

7-5. spread CAUTION (고정)

```
if spread_bps > SPREAD_CAUTION_BPS:
    regimeState = max(regimeState, CAUTION)
    score += (-5)    tag=SPREAD_CAUTION
```

7-6. liquidityState (고정)

```
if volume_m5 <= q(LIQUIDITY_Q_LOW, W_LONG):
    liquidityState = LOW
    score += (-10)    tag=LIQUIDITY_LOW
```

7-7. Regime 점수 (고정)

- OK: 0
- CAUTION: -10 tag=REGIME_CAUTION
- TRANSITION: -20 tag=REGIME_TRANSITION
- HALT: BLOCK

8) CostGate v1.1.2 (고정)

8-1. 하드 차단 (고정)

```
if SC < SC_BLOCK:
    decision = BLOCK(reason=SC_LT_3)
```

8-2. 소프트 감점 (고정)

- $3.0 \leq SC < 4.0 \rightarrow$ score += (-15) tag=SC_MARGINAL
- $4.0 \leq SC < 5.0 \rightarrow$ score += (-5) tag=SC_OK
- $SC \geq 5.0 \rightarrow$ score += (0) tag=SC_GOOD (선택; 의미용)

9) RewardProxyAdjust v1.1.2 (새로 추가; soft only; 고정)

목표:

- SC는 비용 대비 스태폭만 반영한다.
- 기대값 프록시를 완전히 외부 입력(tp/rr)에만 의존하면 공백이 생긴다.
- 따라서 "있으면 사용, 없으면 rewardProxy fallback"으로 항상 동작하게 만든다.
- 이 모듈은 절대 BLOCK하지 않는다 (soft only).

9-1. expectedRR_used 계산 우선순위 (고정)

```
1) signal.expectedRR가 유효하면
    expectedRR_used = signal.expectedRR
2) else if signal.tpRefPrice가 유효하면:
    reward_bps = abs(tpRefPrice - entryRefPrice) / entryRefPrice * 10000
    expectedRR_used = reward_bps / s_effective_bps
3) else
    expectedRR_used = null    (fallback 사용)
```

9-2. expectedRR 기반 패널티 (고정; soft)

```
if expectedRR_used != null:
    if expectedRR_used < 1.0
        -> score += RR_PENALTY_LT_1_0,    tag=RR_LT_1_0
    else if expectedRR_used < 1.2
        -> score += RR_PENALTY_LT_1_2,    tag=RR_LT_1_2
    else
        -> score += 0,                      tag=RR_OK
```

```
else:
    -> tag=RR_UNKNOWN
```

9-3. fallback rewardProxy 패널티 (고정; soft)

expectedRR_used가 null일 때만 적용:

```
rewardProxy == LOW -> score += REWARDPROXY_LOW_PENALTY, tag=REWARDPROXY_LOW
rewardProxy == MID -> score += REWARDPROXY_MID_PENALTY, tag=REWARDPROXY_MID
rewardProxy == HIGH -> score += 0, tag=REWARDPROXY_HIGH
```

10) ConcentrationGate v1.1.2 (패치 유지; 고정)

목표:

- 동일 자리(가격 클러스터)에서 여러 지표가 동시에 올려도 "1개 셋업"으로 취급
- 스팸으로 인해 강한 신호가 역설적으로 차단되는 문제 방지

10-1. poiClusterKey 계산 (고정)

```
clusterStepPrice = midPrice * POI_CLUSTER_STEP_BPS / 10000
clusterStepTicks = max(1, round(clusterStepPrice / tickSize))
entryTicks = round(entryBoundaryPrice / tickSize)
poiClusterKey = floor(entryTicks / clusterStepTicks) (정수)
```

10-2. 카운팅 규칙 (고정)

15분 윈도우 내 같은 (symbol, dir)에 대해:

- unique poiClusterKey 개수만 카운트한다.
- 같은 클러스터에서 재발화(2회 이상)는 duplicate penalty만 적용한다.

duplicate:

```
same (symbol, dir, poiClusterKey) 재발화
-> score += CONC_DUPLICATE_PENALTY, tag=CONC_DUPLICATE
```

10-3. cap 처리 + exceptional override (고정)

```
if uniqueClusters15m >= MAX_UNIQUE_CLUSTERS_15M_PER_DIR:
    if isExceptional == false:
        decision = BLOCK(reason=CONC_TOO_MANY)
    else:
        decision 유지 (ALLOW)
        score += CONC_OVERRIDE_PENALTY
        tag=CONC_OVERRIDE
        reason=CONC_OVERRIDE
```

11) EdgeEvidenceGate v1.1.2 (고정 + coldstart 방어)

목표:

- 장기적으로 음수인 시그니처를 방어
- 초기 데이터 공백은 결정론적으로 처리
- coldstart 기간에는 "작게"로 생존하면서 데이터 축적

11-1. 시그니처 (계층형; 고정)

- sig_coarse = (poiTier, dir, regimeBucket)
- sig_mid = (source, poiTier, dir, regimeBucket)
- sig_fine = (source, poiTier, dir, eventType, regimeBucket)

regimeBucket (고정):

- if regimeState == OK and liquidityState == NORMAL → "OK"
- else if regimeState == TRANSITION → "TRANSITION"
- else → "CAUTION"

11-2. 사용 우선순위 (고정)

```
fine(n >= EDGE_MIN_SAMPLES)
else mid(n >= EDGE_MIN_SAMPLES)
else coarse(n >= EDGE_MIN_SAMPLES)
else NO_EVIDENCE
```

11-3. NO_EVIDENCE 처리 (고정)

- score += EDGE_NO_EVIDENCE_PENALTY tag=EDGE_NO_EVIDENCE
- score += EDGE_COLDSTART_EXTRA_PENALTY tag=EDGE_COLDSTART
- evidenceLevel = NO_EVIDENCE
- BLOCK 금지

suggestedRiskPct는 RiskManager에서 최소 L1 수준으로 클램프한다 (11-3은 "표시/태그/점수", 클램프는 12장)

11-4. lcbR 계산 (고정)

```
lcbR = meanR - LCB_Z * (stdR / sqrt(n))
```

11-5. lcbR < 0 처리 (고정)

하드 차단 조건:

```
lcbR < 0
AND SC < EDGE_LCB_NEG_BLOCK_SC
AND collabStrength != STRONG
AND isExceptional == false
-> decision = BLOCK(reason=EDGE_LCB_NEG_BLOCK)
```

예외 (차단 대신 강감점; 고정):

```
그 외 lcbR < 0
-> score += EDGE_LCB_NEG_PENALTY, tag=EDGE_LCB_NEG_OVERRIDE
```

11-6. Kelly 힌트 (조건부; 기본 OFF)

- KELLY_DEFAULT_ENABLED=false 인 동안 출력하지 않는다.

12) RiskManager v1.1.2 (HALT hysteresis + coldstart clamp 포함; 고정)

12-1. HALT hysteresis (고정)

AccountSnapshot.riskMode(이전 상태)를 입력으로 사용한다.

```
if prevRiskMode != HALT:
    if realizedPnl_24h_pct <= HALT_ENTER_PNL_24H:
        nextRiskMode = HALT

else (prevRiskMode == HALT):
    if realizedPnl_24h_pct > HALT_EXIT_PNL_24H:
        nextRiskMode = (L2/L1/NORMAL 재평가)
    else:
        nextRiskMode = HALT
```

HALT이면:

- decision = BLOCK(reason=RISK_HALT_ROLLING_24H)
- suggestedRiskPct = 0

12-2. 연속 손실 기반 모드 (HALT가 아니면 적용)

- consecutiveLosses >= CONSEC_LOSS_L2 → L2
- consecutiveLosses >= CONSEC_LOSS_L1 → L1
- else → NORMAL

12-3. 조기 복귀 최소 R (고정)

L1 → NORMAL 조건:

- lastWinR_afterCost >= MIN_RECOVERY_R_L1

L2 → L1 조건:

- sum(last2WinsR_afterCost) >= MIN_RECOVERY_R_L2_SUM2WINS
- 승리 기록이 없으면 복귀 불가.

12-4. 모드별 suggestedRiskPct 기본값 (고정; 출력만)

- NORMAL → 0.010
- L1 → 0.006
- L2 → 0.003
- HALT → 0.000

12-5. coldstart risk clamp (고정)


```
if evidenceLevel == NO_EVIDENCE:
    suggestedRiskPct = min(suggestedRiskPct, RISK_PCT_L1)
```

12-6. RiskManager 점수 (고정)

- NORMAL: 0
- L1: score += (-15) tag=RISK_L1
- L2: score += (-25) tag=RISK_L2
- HALT: BLOCK

13) PortfolioExposureGate v1.1.2 (새로 추가; non-overridable; 고정)

목표 및 설계 근거:

- 포트폴리오 총 리스크(동시 노출)가 cap를 넘는 순간, 신호 품질과 무관하게 계좌 꼬리위험이 폭증한다.
- ConcentrationGate의 exceptional은 "스캠 역설"을 풀기 위한 것이지만, PortfolioExposure는 "계좌 전체가 동시에 터지는 실패모드"를 막는 장치다.
- 따라서 PortfolioExposure는 exceptional override를 절대 허용하지 않는다.

13-1. 입력 존재 여부 (고정)

```
if account.openRiskPct가 없으면:
    Gate를 SKIP한다 (BLOCK/trim/penalty 없음)
    tag=PORTFOLIO_UNKNOWN (선택; 운영 가시성)
```

13-2. cap 선택 (고정)

```
cap = by riskMode:
    NORMAL -> PORTFOLIO_CAP_NORMAL (0.020)
    L1      -> PORTFOLIO_CAP_L1     (0.012)
    L2      -> PORTFOLIO_CAP_L2     (0.006)
    HALT    -> PORTFOLIO_CAP_HALT   (0.000)
```

13-3. 규칙 (고정; non-overridable)

```
if openRiskPct >= cap:
    decision = BLOCK(reason=PORTFOLIO_FULL)
    suggestedRiskPct = 0

else if openRiskPct + suggestedRiskPct > cap:
    decision 유지 (ALLOW)
    suggestedRiskPct = cap - openRiskPct (risk trim)
    tag=PORTFOLIO_TRIMMED
    reason=PORTFOLIO_TRIMMED
    (선택) score += (-5) tag=PORTFOLIO_TRIM_PENALTY

else:
    변화 없음
```

14) 최종 합산 규칙 (고정)

14-1. policyScoreDeltaSum (고정)

```
policyScoreDeltaSum = sum(all score deltas from gates)
```

14-2. tags/reasons 정렬 (고정)

```
policyTags = unique(tags) -> lexicographic sort
reasons     = unique(reasons) -> lexicographic sort
```

15) BLOCK 우선순위 (고정)

아래 중 하나라도 만족하면 decision=BLOCK :

- 1) missing_required_field
- 2) RegimeGate HALT
- 3) CostGate SC < 3.0
- 4) RiskManager HALT (hysteresis 포함)
- 5) ConcentrationGate cap 초과 (단, exceptional이면 BLOCK 금지)
- 6) EdgeEvidenceGate lcbR<0 hard block 조건

16) 구현 주의사항 (고정)

- 1) Derived pool은 반드시 1회만 계산 후 모든 gate가 공유한다 (중복 계산 금지).
 - 2) entryRefPrice = midPrice else lastPrice를 강제한다.
 - 3) isExceptional은 강한 자리 보존 목적이거나, Regime HALT / SC<3 / Risk HALT / Portfolio cap은 override 불가.
 - 4) signalsSent_60m은 Router-only이며 Policy는 사용하지 않는다.
 - 5) rewardProxy는 RR의 "대체 입력"이 아니라, RR 입력이 없을 때만 쓰는 기대값 프록시다 (soft only).
-

17) v1.1.1 → v1.1.2 통합 패치 요약 (정본 내장)

- (HALT 명세 확정) rolling 24h 기반 stateless + hysteresis(ENTER=-2.0, EXIT=-1.8)
 - (TP/RR 공백 해결) expectedRR/tpRefPrice 있으면 사용, 없으면 poiTier+SC rewardProxy fallback(soft only)
 - (Router 책임 분리) signalsSent_60m은 Router-only로 명시(Policy optional)
 - (Coldstart 방어) NO_EVIDENCE에서 risk clamp(L1) + -10 + EDGE_COLDSTART
 - (포트폴리오 생존) PortfolioExposureGate 추가, exceptional override 금지, cap 초과 시 block/trim
-

DEV SPEC — 데이터모델 / 상태머신 / 테스트 커버리지

POLICY LAYER v1.1.2 개발용 부록

A) 권장 폴더 구조 (고정 권장)

```
lib/engine/policy/
├─ constants.ts      (정본 3장 상수 1:1)
├─ types.ts          (SignalCandidate/MarketSnapshot/AccountSnapshot/PolicyResult/DerivedValues)
├─ derived.ts        (computeDerivedValues: 단 1회)
├─ policy.ts         (PolicyEvaluate: 파이프라인 순서 고정)
├─ gates/
│  └─ regime.ts
│  └─ cost.ts
│  └─ rewardProxy.ts (RewardProxyAdjust)
│  └─ concentration.ts
│  └─ edge.ts
│  └─ risk.ts        (HALT hysteresis 포함)
│  └─ portfolio.ts   (PortfolioExposureGate)
├─ stores/
│  └─ regimeStore.ts (W_LONG/W_SHORT ring buffer + quantile/mean)
│  └─ concStore.ts  (15m cluster history)
│  └─ edgeStore.ts  (signature stats: meanR/stdR/n)
│  └─ riskStore.ts  (lastWinR, last2WinsSumR)
└─ tests/
   └─ policy.e2e.spec.ts
   └─ regime.spec.ts
   └─ cost.spec.ts
   └─ rewardProxy.spec.ts
   └─ concentration.spec.ts
   └─ edge.spec.ts
   └─ risk.spec.ts
   └─ portfolio.spec.ts
```

B) 데이터 모델 (types.ts) — 필수/선택 필드 고정

Enums

- Decision: "ALLOW" | "BLOCK"
- Dir: "BULL" | "BEAR"
- Source: "FVG" | "OB" | "CHANNEL" | "TRENDLINE"
- EventType: "REACTION" | "ENTRY_WINDOW_OPEN"
- PoiTier: "D1_POI" | "H4_CORE" | "SETUP" | "OTHER"
- RiskMode: "NORMAL" | "L1" | "L2" | "HALT"
- RegimeState: "OK" | "CAUTION" | "TRANSITION" | "HALT"
- VolState: "LOW" | "NORMAL" | "HIGH"
- LiquidityState: "NORMAL" | "LOW"
- CollabStrength: "NONE" | "WEAK" | "STRONG"
- EvidenceLevel: "FINE" | "MID" | "COARSE" | "NO_EVIDENCE"
- RewardProxy: "HIGH" | "MID" | "LOW"

필드 처리 원칙

- AccountSnapshot.openRiskPct: optional (없으면 PortfolioExposureGate SKIP)
- AccountSnapshot.signalsSent_60m: optional, Router-only (Policy ignores)

C) RiskMode 상태머신 (HALT hysteresis 포함)

입력

- prevRiskMode (AccountSnapshot.riskMode)
- realizedPnl_24h_pct
- consecutiveLosses
- riskStore (승리 기록)

전이 (결정론)

- 1) HALT hysteresis

- prev != HALT and pnl <= ENTER → HALT
- prev == HALT and pnl > EXIT → exit HALT and continue evaluate
- prev == HALT and pnl <= EXIT → HALT 유지

2) (HALT가 아니면) consecutiveLosses → L2/L1/NORMAL

3) 조기복귀: riskStore 기반

From	Guard(트리거)	To	Notes
ANY	prev!=HALT and pnl <= -2.0	HALT	하드 차단, override 불가
HALT	pnl > -1.8	재평가(L2/L1/NORMAL)	hysteresis 해제
HALT	pnl <= -1.8	HALT 유지	플래핑 방지
NORMAL	consecutiveLosses >= 5	L2	HALT 우선
NORMAL	consecutiveLosses >= 3	L1	HALT/L2 우선
L1	consecutiveLosses >= 5	L2	악화
L1	lastWinR_afterCost >= 0.30	NORMAL	조기 복귀
L2	sum(last2WinsR_afterCost) >= 0.40	L1	조기 완화
L2	복귀 조건 불충족	L2 유지	승리 기록 없으면 유지

D) 파이프라인 단계별 변경 허용 범위

단계	파일	변경 허용
1	derived.ts	DerivedValues만 계산 (1회)
2	gates/regime.ts	decision/scoreDelta/tags/reasons 업데이트
3	gates/cost.ts	decision/scoreDelta/tags/reasons
4	gates/rewardProxy.ts	scoreDelta/tags 만 (BLOCK 금지)
5	gates/concentration.ts	decision(단 exceptional은 BLOCK금지)/scoreDelta/tags/reasons
6	gates/edge.ts	decision/scoreDelta/tags/reasons
7	gates/risk.ts	riskMode/suggestedRiskPct 산출, coldstart clamp 적용
8	gates/portfolio.ts	suggestedRiskPct trim 또는 block (override 금지)
9	policy.ts (finalize)	tags/reasons unique+sort

E) 테스트 커버리지 매트릭스 (최소 26개 권장)

TC	시나리오	기대 결과	타겟
TC01	required field 누락	BLOCK(missing_required_field)	policy.ts
TC02	spread_bps > 50	BLOCK(REGIME_SPREAD_HALT)	regime.ts
TC03	shiftWeak	CAUTION 및 -10	regime.ts
TC04	shiftStrong	TRANSITION 및 -20	regime.ts
TC05	liquidity LOW	-10	regime.ts
TC06	SC < 3	BLOCK(SC_LT_3)	cost.ts
TC07	SC 3~4	-15	cost.ts
TC08	expectedRR < 1.0	-15, RR_LT_1_0	rewardProxy.ts
TC09	tpRefPrice로 RR 계산	expectedRR_used != null	rewardProxy.ts
TC10	RR unknown + rewardProxy LOW	-10	rewardProxy.ts
TC11	rewardProxy HIGH	0	rewardProxy.ts
TC12	concentration 동일 cluster multi-confirm	unique 1	concentration.ts
TC13	concentration duplicate	-10	concentration.ts
TC14	concentration 6번째 cluster	BLOCK	concentration.ts
TC15	concentration limit + exceptional	BLOCK 금지, -30, CONC_OVERRIDE	concentration.ts
TC16	edge NO_EVIDENCE	-5 + -10, EDGE_COLDSTART	edge.ts

TC17	coldstart clamp	suggestedRiskPct <= 0.006	risk.ts
TC18	lcbR<0 hard block 조건	BLOCK(EDGE_LCB_NEG_BLOCK)	edge.ts
TC19	lcbR<0 but strong	-35, BLOCK 금지	edge.ts
TC20	HALT enter: prev!=HALT, pnl<=-2.0	HALT BLOCK	risk.ts
TC21	HALT 유지: prev==HALT, pnl<=-1.8	HALT 유지 BLOCK	risk.ts
TC22	HALT exit: prev==HALT, pnl>-1.8	정상 모드 재평가	risk.ts
TC23	portfolio openRiskPct missing	SKIP + PORTFOLIO_UNKNOWN(선택)	portfolio.ts
TC24	portfolio full: open>=cap	BLOCK(PORTFOLIO_FULL)	portfolio.ts
TC25	portfolio trim: open+new>cap	ALLOW, suggestedRiskPct trimmed	portfolio.ts
TC26	tags/reasons unique+sort 결정론	결정론 출력	policy.ts finalize

"행 수 > 10이면 테스트 증가" 원칙을 Policy에도 적용하면, 위 26개는 딱 최소치다.

F) 구현 LOCK 10줄 (가장 자주 꼬이는 부분만)

- 1) derived는 1회 계산 (중복 spread 계산 금지)
- 2) entryRefPrice = midPrice else lastPrice
- 3) HALT는 hysteresis ENTER/EXIT 적용 (입력 prevRiskMode 필요)
- 4) SC<3, Regime HALT, Risk HALT, Portfolio FULL은 override 금지
- 5) Concentration은 poiClusterKey로 그룹핑 (동일 자리 multi-confirm count=1)
- 6) Concentration cap 초과는 exceptional이면 block 대신 -30
- 7) Edge NO_EVIDENCE는 -5 + -10 + coldstart risk clamp(L1)
- 8) RewardProxyAdjust는 절대 BLOCK 금지 (soft only)
- 9) PortfolioExposure는 exceptional override 금지, cap 초과 시 block/trim
- 10) tags/reasons는 unique + lexicographic sort

목적

- 원문(policy_layer_v1.1.2.pdf)은 "수정/삭제" 없이 그대로 유지한다.
- 아래 항목을 '추가(LOCK)'로만 적용한다. (결정론/재현성 우선)

A) 입력: MarketSnapshot — data integrity 필드 추가 (LOCK)

추가 필드

- dataOk: boolean (필수)
- dataState: OK|BACKFILLING|GAP_DETECTED (선택; 권장)
- dataGapBars: number (선택; 권장)
- dataGapFromTime: ISO (선택; 권장)

정의(LOCK)

- dataOk = true $\Leftrightarrow$ 현재 시각(time) 기준,
관측 TF(M5/M15/M30/H1/H4/D1)에서 "누락 캔들 0" AND "backfill 미진행".
- dataOk = false $\Leftrightarrow$ 위 조건 불만족(WS 재연결/24h 강제 종료/REST 보충 중/순서 미확정 등).
- dataState를 제공하는 경우, dataOk와 반드시 정합해야 한다.
 - * dataOk=true $\rightarrow$ dataState=OK
 - * dataOk=false $\rightarrow$ dataState $\in$ {BACKFILLING, GAP_DETECTED}

B) DataIntegrityGate 추가 (파이프라인 최상단, LOCK)

위치(LOCK)

- CostGate 이전(모든 Gate/Score 계산보다 먼저)

규칙(LOCK)

- ```
if market.dataOk == false:
```
- decision = BLOCK
  - reasons += ["DATA\_INTEGRITY"]
  - policyTags += ["DATA\_GAP"]
  - score = 0
  - 이후 Gate/Score 계산은 SKIP (결정론: 위 결과만 반환)

### 오버라이드(LOCK)

- isExceptional은 DataIntegrityGate를 절대 override할 수 없다.  
(PortfolioExposureGate / Regime=HALT / SC<3.0과 동일한 '하드 방어' 성격)

## C) 가격 기준(Price Basis) LOCK — MarkPrice는 v1.1.2 범위 밖

### LOCK

- PolicyLayer의 market.bid/ask/last/mid는 "Last Price(거래가) 기반" 시세를 사용한다.
- atr/rv/volume 등 candle 파생값도 Indicator 엔진과 동일한 kline>Last Price) 기반으로 계산된 값을 입력.
- Mark Price / Index Price는 v1.1.2 범위 밖(Non-goal)이다.  
(향후 버전에서만 추가/전환 가능)

## D) Funding Rate (Non-goal) — 명시 추가

### LOCK

- v1.1.2는 스캘핑/단타 운영을 전제로 fundingRate 기반 BLOCK/패널티를 두지 않는다.  
(향후 필요 시 v1.2+에서 별도 도입)